

Funktioner som parametrar

En aspekt av C som inte används speciellt ofta i C++ är möjligheten att skicka funktioner som parametrar till andra funktioner. Detta möjliggör en form av generisk programmering där man kan skapa funktioner vars beteende kan ändras genom att olika funktioner skickats som parameter appliceras på någon form av data.

I C++ kan man för det mesta använda andra metoder för att uppnå samma funktionalitet, t.ex. med hjälp av klasser (se [Kapitel 14](#)).

Funktionsvariabler

Man kan skapa variabler som innehåller *adressen* för en funktion. Dessa variabler kan sedan skickas som parametrar till andra funktioner, för att sedan i något skede anropas. Allmänt definierar man en variabel som kan "innehålla" eller peka på (se [Kapitel 9](#)) en funktion på följande sätt:

```
returtyp (*variabelnamn) (parametertyp1, parametertyp2, ...);
```

Variabeln måste vara definierad exakt på samma sätt som den funktion man vill att den skall kunna innehålla. Vill man ha en variabel som innehåller en funktion som returnerar en `int` och tar som parameter två `string` skall den definieras på följande sätt:

```
int (*variabelnamn) (string, string);
```

Ifall funktionen inte returnerar någonting skall dess returtyp vara `void`, och ifall den inte tar några parametrar skall parameterlistan vara tom. Parentesen och asterisken (*) runt variabelnamnet indikerar att det är frågan om en pekare till en funktion.

Innan en funktionsvariabel kan användas måste den tilldelas adressen för en funktion. Vi kan t.ex. ha en funktion som tar en `int` som argument och returnerar en annan `int`. Vi kan då tilldela denna funktion till en funktionsvariabel på följande sätt:

```
// funktion som dubblar ett tal
int dubblar (int Tal) {
    return Tal + Tal;
}

// variabel för funktion
int (*funktion)(int);

// tilldela variabeln
funktion = dubblar;
```

En alternativ syntax för själva tilldelningen är:

```
// tilldela variabeln
funktion = &dubblar;
```

Där ger man explicit adressen för funktionen. Båda sätten är ekvivalenta.

Anropa funktionsvariabler

Om man har en variabel som innehåller en funktion kan den användas som vilken normal funktion som helst. Om vi ser på exemplet ovan med en variabel som heter `funktion` kan den anropas på följande sätt:

```
// anropa funktionsvariabel
int Resultat = funktion (10);
```

Man säger att man *avrefererar* funktionsvariabeln. Ett lite mera omfattande exempel på hur funktionsvariabler kan användas kommer nedan.

```
#include <iostream>
#include <iomanip>

// funktion som dubblar ett tal
int dubblara (int Tal) {
    return Tal + Tal;
}

// funktion som kvadrerar ett tal
int kvadrera (int Tal) {
    return Tal * Tal;
}

// funktion som applicerar en given funktion på en serie med tal
void applicera (int (*funktion)(int) ) {
    // iterera över talen 0 till 4
    for (int Index = 0; Index <= 4; Index++ ) {
        // applicera den givna funktionen på talet
        cout << "Tal: " << setw(3) << Index << " blir: "
             << setw(4) << funktion(Index) << endl;
    }
}

int main () {
    cout << "Funktion: dubblara" << endl;

    // anropa applicera() med funktionen dubblara()
    applicera ( dubblara );

    cout << endl << "Funktion: kvadrera" << endl;

    // anropa applicera() med funktionen kvadrera()
    applicera ( kvadrera );
}
```

Här skickas funktionerna `dubblara()` och `kvadrera()` som parametrar till `applicera()` i tur och ordning. Inne i `applicera()` används parametern `funktion` som vilken normal funktion som helst som tar som parameter en `int` och returnerar en `int`. På detta sätt kan man skapa generiska funktioner som utför någon viss beräkning på en serie tal.

Notera att då vi skickar funktionerna som parametrar skriver vi *inte* ut tomma paranteser efter namnen, utan endast funktionens namn.

Körning av programmet ovan ger följande utskrift:

```
% ./FunktionsParameterar
```

Funktion: dubblering

Tal: 0 blir: 0

Tal: 1 blir: 2

Tal: 2 blir: 4

Tal: 3 blir: 6

Tal: 4 blir: 8

Funktion: kvadrering

Tal: 0 blir: 0

Tal: 1 blir: 1

Tal: 2 blir: 4

Tal: 3 blir: 9

Tal: 4 blir: 16

%

[Förutgående](#)

Rekursion

[Hem](#)

[Upp](#)

[Nästa](#)

Input och output